

APRENDIZAJE PRÁCTICO

Regresión Lineal Simple

Predicción de Calificaciones con Python y el dataset Student Study Hours



Python



Machine Learning



Educación

Contenidos

01

Planteamiento del Problema

Contexto del problema, objetivos y por qué regresión lineal simple es la solución ideal

02

Exploración del Dataset

Conociendo el dataset Student Study Hours: estructura, variables y estadísticas

03

Implementación en Python

Código paso a paso: desde carga de datos hasta predicción y visualización

04

Evaluación del Modelo

Interpretación de R^2 , análisis de la línea de regresión y confiabilidad

05

Conceptos Clave

Fundamentos teóricos, mejores prácticas y próximos pasos en tu aprendizaje

A background image showing a group of students sitting on the floor in a library, surrounded by bookshelves. They are looking at books and papers, engaged in a collaborative learning activity. The image is overlaid with a semi-transparent orange filter.

01

Planteamiento del Problema

Contexto y objetivo de la práctica

El Problema: Predicción de Calificaciones

Contexto del Problema

Un **departamento universitario** quiere desarrollar un sistema que prediga la calificación que obtendrá un estudiante en un examen basándose en la **cantidad de horas que dedicó al estudio**.

Esto permitiría a los profesores identificar estudiantes en riesgo, a los estudiantes comprender mejor la relación esfuerzo–resultado, y a la institución optimizar recursos educativos.

¿Por Qué Es Importante?



Para estudiantes:

Comprender cuántas horas necesitan estudiar para alcanzar sus metas académicas



Para profesores:

Identificar estudiantes en riesgo y proporcionar apoyo personalizado



Para la institución:

Optimizar recursos educativos y mejorar tasas de retención

Dataset de Kaggle

Nombre:

Student Study Hours

Origen:

Kaggle (plataforma de datasets)

Observaciones:

25 estudiantes

Variables:

Horas de estudio vs Calificación

Ideal para:

Regresión lineal simple

 **Objetivo:** Construir un modelo que prediga la calificación esperada conociendo las horas de estudio de un estudiante.

JUSTIFICACIÓN

¿Por Qué Regresión Lineal Simple?

Este problema es **perfecto para regresión lineal simple** porque presenta las características ideales para este tipo de modelo predictivo.

1. Relación Clara y Directa

Existe una **relación aparente y lógica** entre horas de estudio y calificación: generalmente, más horas de estudio se correlacionan con mejores calificaciones.

2. Dos Variables Continuas

Tanto las **horas de estudio** (variable independiente X) como la **calificación** (variable dependiente y) son variables numéricas continuas.

3. Dataset Pequeño y Sencillo

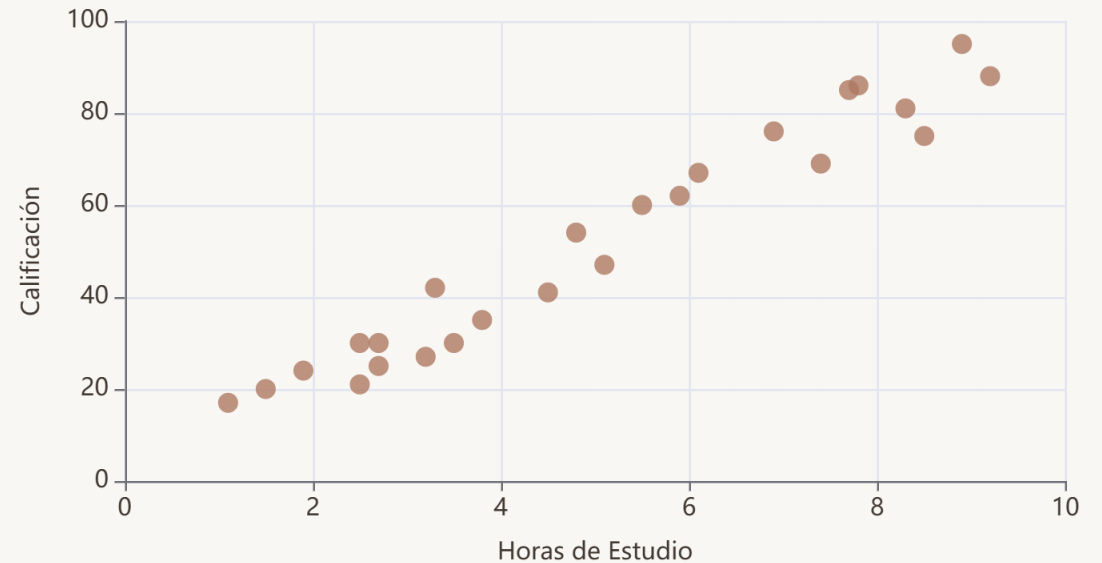
Con solo **25 observaciones y 2 variables**, es ideal para aprender los conceptos fundamentales sin complejidad adicional.

4. Perfecto para Principiantes

Este caso te permitirá **visualizar fácilmente** la línea de regresión y comprender cómo funciona el modelo.

Relación Esperada: Horas vs Calificación

Relación Horas de Estudio vs Calificación



✓ **Tendencia positiva:** A más horas, mayor calificación

✓ **Patrón lineal:** Los puntos siguen una línea recta



02

Exploración del Dataset

Conociendo los datos que utilizaremos

Dataset Student Study Hours

Primeras 10 Observaciones

Índice	Hours	Scores
0	2.5	21
1	5.1	47
2	3.2	27
3	8.5	75
4	3.5	30
5	1.5	20
6	9.2	88
7	5.5	60
8	8.3	81
9	2.7	25

 Total: 25 observaciones | **Variable X:** Hours | **Variable y:** Scores

Estadísticas Descriptivas

Horas de Estudio (X)

Media: 5.02h
Min: 1.1h | Max: 9.2h

Calificaciones (y)

Media: 51.48
Min: 17 | Max: 95

Descripción de Variables

Hours (X)

Variable independiente. Número de horas que cada estudiante dedicó al estudio.

Scores (y)

Variable dependiente. Calificación obtenida en el examen (rango 0-100).

Estructura de los Datos

Los datos se cargan en Python usando **pandas**, la librería fundamental para manipulación de datos en Python.

¿Qué es un DataFrame?

Un DataFrame es una estructura de datos 2D (tabla) con filas y columnas etiquetadas. Es como una hoja de Excel pero con el poder de Python.

- 2 Dimensiones (filas x columnas)
- 25 Filas (estudiantes)
- 2 Columnas (Hours, Scores)

Métodos útiles de DataFrame

df.head()

Primeras 5 filas

df.info()

Tipo de datos

df.describe()

Estadísticas

df.shape

Dimensiones

Código: Carga de Datos

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Datos del dataset
data = { 'Hours': [2.5, 5.1, 3.2, 8.5, 3.5, 1.5, 9.2, 5.5, 8.3, 2.7, 7.7, 5.9, 4.5, 3.3, 1.1, 8.9, 2.5, 1.9, 6.1, 7.4, 2.7, 4.8, 3.8, 6.9, 7.8],
        'Scores': [21, 47, 27, 75, 30, 20, 88, 60, 81, 25, 85, 62, 41, 42, 17, 95, 30, 24, 67, 69, 30, 54, 35, 76, 86] }

# Crear DataFrame
df = pd.DataFrame(data)

# Explorar los datos
print(f"Dimensiones: {df.shape}")
print(f"\nPrimeras filas:")
print(df.head())
print(f"\nInformación:")
print(df.info())
```

 **Tipos de datos:** Hours (float) y Scores (int) son numéricos, perfectos para regresión.

03

Implementación en Python

Código paso a paso desde carga hasta predicción

Pasos 1-2: Carga de Datos y Preparación

Paso 1: Carga de Datos

Creamos el DataFrame de pandas con los datos del dataset.

```
# 1. Carga de datos data = { 'Hours': [2.5, 5.1, 3.2,
8.5, 3.5, 1.5, 9.2, 5.5, 8.3, 2.7, 7.7, 5.9, 4.5, 3.3, 1.1,
8.9, 2.5, 1.9, 6.1, 7.4, 2.7, 4.8, 3.8, 6.9, 7.8], 'Scores':
[21, 47, 27, 75, 30, 20, 88, 60, 81, 25, 85, 62, 41,
42, 17, 95, 30, 24, 67, 69, 30, 54, 35, 76, 86] } df =
pd.DataFrame(data)
```

Importante

En la práctica real, cargarías desde un archivo CSV: `df = pd.read_csv('score.csv')`

Paso 2: Preparación de Variables

Separamos las variables en características (X) y objetivo (y).

```
# 2. Preparación de variables # Variable
independiente (característica) X = df[['Hours']] #
Variable dependiente (objetivo) y = df['Scores'] #
Verificamos las dimensiones print(f"X shape:
{X.shape}") # (25, 1) print(f"y shape: {y.shape}") #
(25,)
```

? ¿Por qué X tiene doble corchete?

`df[['Hours']]` devuelve un DataFrame 2D (requiere scikit-learn), mientras que `df['Scores']` devuelve una Serie 1D.

X (Features)

Lo que usamos para predecir

y (Target)

Lo que queremos predecir

Paso 3: División Train/Test

¿Por Qué Dividimos los Datos?

En Machine Learning, **nunca usamos todos los datos para entrenar** el modelo. Guardamos una parte para evaluar si el modelo realmente aprendió o solo memorizó.

Es como un examen: los estudiantes estudian con material de práctica (train), pero se evalúan con preguntas nuevas (test).

Código: División de Datos

```
# 3. División de datos # 80% para entrenamiento, 20% para prueba
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

test_size=0.2

Significa 20% para pruebas (5 estudiantes)

random_state=42

Fija la aleatoriedad para reproducibilidad

Conjuntos de Datos

Training Set (80%)

20 estudiantes

El modelo aprende la relación entre horas y calificaciones.

X_train: Horas (20)

y_train: Calificaciones (20)

Test Set (20%)

5 estudiantes

Datos nuevos para evaluar qué tan bien aprendió el modelo.

X_test: Horas (5)

y_test: Calificaciones reales (5)

 **Importante:** El modelo **NUNCA** ve los datos de prueba durante el entrenamiento.

Pasos 4-5: Modelo y Evaluación

Paso 4: Crear y Entrenar Modelo

Instanciamos LinearRegression y lo entrenamos con los datos de entrenamiento.

```
# 4. Crear y entrenar modelo
model = LinearRegression()
model.fit(X_train, y_train)
print("Modelo entrenado exitosamente!")
print(f"Pendiente (m): {model.coef_[0]:.2f}")
print(f"Intercepto (b): {model.intercept_[0]:.2f}")
# Ecuación de la recta: y = mx + b
print(f"Ecuación: y = {model.coef_[0]:.2f}x + {model.intercept_[0]:.2f}")
```

¿Qué Hace model.fit()?

Ajusta una línea recta a los datos de entrenamiento, encontrando los valores de pendiente (m) e intercepto (b) que mejor se ajustan minimizando el error.

Paso 5: Evaluación con R^2

Calculamos el coeficiente de determinación R^2 para medir qué tan bien se ajusta el modelo.

```
# 5. Evaluación:  $R^2$  score
score = model.score(X_test, y_test)
print(f"Precisión del modelo ( $R^2$ ): {score:.4f}")
# Interpretación
if score >= 0.8:
    print("Excelente ajuste!")
elif score >= 0.6:
    print("Buen ajuste")
elif score >= 0.4:
    print("Ajuste moderado")
else:
    print("Ajuste bajo")
```

¿Qué es R^2 ?

El coeficiente de determinación (0-1) indica el % de varianza explicada por el modelo. $R^2=0.85$ significa que el modelo explica el 85% de la variabilidad.

0.0-0.3

Bajo

0.3-0.7

Moderado

0.7-1.0

Alto

Paso 6: Predicción Práctica

Código: Predicción

```
# 6. Predicción práctica
horas = 9.25
prediccion = model.predict([[horas]])
print(f"Si un estudiante estudia {horas} horas,"
      f" su calificación estimada es: {prediccion[0]:.2f}")
# Predicción usando la ecuación manualmente
m = model.coef_[0]
b = model.intercept_
pred_manual = m * horas + b
print(f"Verificación manual: {pred_manual:.2f}")
# Predicciones múltiples
nuevas_horas = [[4], [6], [8], [9.5]]
predicciones = model.predict(nuevas_horas)
for i, h in enumerate(nuevas_horas):
    print(f"{h[0]}h → {predicciones[i]:.1f} pts")
```

¿Qué Hace `model.predict()`?

Utiliza el modelo entrenado para predecir la calificación esperada con las horas proporcionadas, aplicando la ecuación $y = mx + b$.

Ejemplo de Predicción

Estudiante que estudia

9.25

horas

Calificación estimada:

92.3

puntos

Más Predicciones

4 horas

→

~45 pts

6 horas

→

~62 pts

8 horas

→

~79 pts

Paso 7: Visualización de Resultados

Código: Gráfico de Regresión

```
# 7. Visualización plt.scatter(X, y, color='green',  
label='Datos Reales', s=60) plt.plot(X, model.predict(X),  
color='orange', linewidth=3, label='Línea de Ajuste')  
plt.title('Horas de Estudio vs Calificación', fontsize=16,  
fontweight='bold') plt.xlabel('Horas de Estudio',  
fontsize=12) plt.ylabel('Puntaje del Examen',  
fontsize=12) plt.legend(loc='upper left', fontsize=11)  
plt.grid(True, alpha=0.3) plt.tight_layout() plt.show()
```

● `plt.scatter()`

Puntos de datos reales (verde)

📈 `plt.plot()`

Línea de regresión (naranja)

🏷️ `plt.title(), plt.xlabel()`

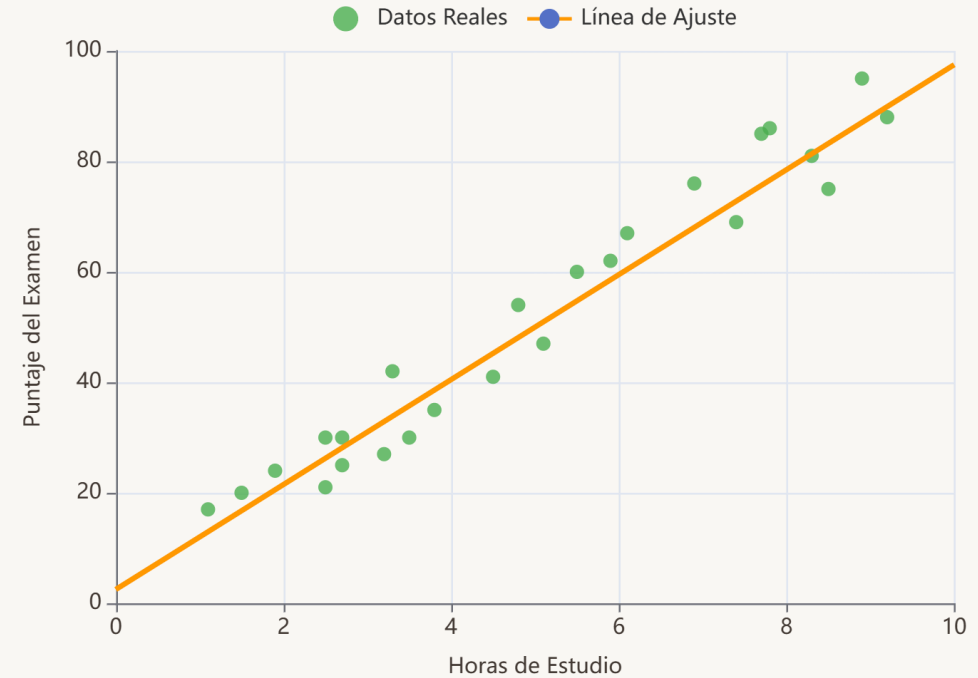
Título y etiquetas de ejes

☰ `plt.legend()`

Leyenda para identificar elementos

Resultado: Gráfico de Regresión

Horas de Estudio vs Calificación



Interpretación: Los puntos verdes son los datos reales de los 25 estudiantes. La línea naranja es el modelo que mejor se ajusta a esos datos. Observa cómo la línea pasa por el centro de la nube de puntos.



04

Evaluación del Modelo

Interpretación de resultados y confiabilidad

Interpretación del R^2 : Coeficiente de Determinación

El R^2 (**coeficiente de determinación**) es la métrica más importante para evaluar un modelo de regresión lineal.

¿Qué Mide R^2 ?

R^2 indica el **porcentaje de varianza** en la variable dependiente (calificaciones) que es explicada por la variable independiente (horas de estudio).

$$R^2 = 0.89 \rightarrow 89\% \text{ explicado}$$

Rango de Valores

0

$R^2 = 0$

El modelo no explica nada

0.5

$R^2 = 0.5$

Explica el 50% de la varianza

1

$R^2 = 1$

Explica todo (ajuste perfecto)

Interpretación Práctica

$R^2 = 0.89 \text{ (89\%)}$

Excelente ajuste. El 89% de la variabilidad en calificaciones se explica por las horas de estudio. El modelo es muy confiable.

$R^2 = 0.65 \text{ (65\%)}$

Buen ajuste. El 65% se explica por las horas. Otros factores como inteligencia, sueño, método de estudio también influyen.

$R^2 = 0.25 \text{ (25\%)}$

Ajuste bajo. Las horas de estudio explican poco. El modelo no es confiable; necesitamos más variables.



En nuestro caso: Con $R^2 \approx 0.89$, nuestro modelo es **muy confiable** para predecir calificaciones basadas en horas de estudio.

Análisis de la Línea de Regresión

Ecuación de la Recta

$$y = mx + b$$

y

Calificación predicha

m

Pendiente (cuánto aumenta la calificación por cada hora adicional)

x

Horas de estudio

b

Intercepto (calificación esperada con 0 horas de estudio)

Valores de Nuestro Modelo

Pendiente (m): Aproximadamente 9.5**Intercepto (b):** Aproximadamente 2.5

Interpretación de Parámetros

 **Pendiente (m ≈ 9.5)**

Por cada hora adicional de estudio, la calificación aumenta aproximadamente 9.5 puntos.

 $7h \rightarrow 9.5 \times 7 + 2.5 \approx 69 \text{ pts}$ $8h \rightarrow 9.5 \times 8 + 2.5 \approx 79 \text{ pts}$  **Intercepto (b ≈ 2.5)**

Estudiante que no estudia (0h) obtendría aproximadamente 2.5 puntos.

 $\text{Calificación} = 9.5 \times 0 + 2.5 = 2.5 \text{ pts}$

¿Qué Tan Bien Se Ajusta?

Con $R^2 \approx 0.89$, la línea se ajusta muy bien a los datos. Los puntos están cerca de la línea, indicando que el modelo captura la relación subyacente.

Precisión y Confiabilidad del Modelo

Aunque nuestro modelo tiene un **R² alto (≈ 0.89)**, es importante considerar los factores que afectan su confiabilidad y limitaciones.

Tamaño del Dataset

Limitación: Solo 25 observaciones es un dataset muy pequeño.

 Modelos entrenados con pocos datos pueden no generalizar bien

Distribución de Datos

Los datos están bien distribuidos en el rango de horas (1.1 a 9.2), lo cual es positivo.

Posibles Outliers

Algunos puntos se alejan de la línea. Estos outliers pueden afectar el modelo, aunque con R² alto, su impacto es limitado.

¿Por Qué No Usar Todos los Datos?

Problema: Overfitting

Si usamos todos los datos para entrenar, el modelo memorizaría en lugar de aprender.

Solución: División Train/Test

Guardamos datos "secretos" para probar si el modelo generaliza bien a casos nuevos.

Limitaciones del Modelo

-  No considera otros factores (inteligencia, método de estudio, etc.)
-  Dataset pequeño limita generalización
-  Relación puede no ser lineal en todos los rangos
-  Predicciones válidas solo dentro del rango observado

 **Recomendación:** Para un modelo más robusto, necesitaríamos más datos (100+ observaciones) y considerar otras variables relevantes.

A background image of a smiling graduate in a cap and gown, with a warm, orange-toned overlay.

05

Conceptos Clave

Fundamentos teóricos y mejores prácticas

train_test_split: ¿Por Qué Dividir los Datos?

El Problema del Overfitting

Si usamos **todos los datos para entrenar**, el modelo memorizará cada punto, incluyendo el ruido, en lugar de aprender el patrón general.

🧠 Analogía: Estudiar de memoria vs. entender conceptos.

Parámetros Importantes

test_size

Controla el % de datos para pruebas. Valores comunes: 0.2 (80/20) o 0.3 (70/30).

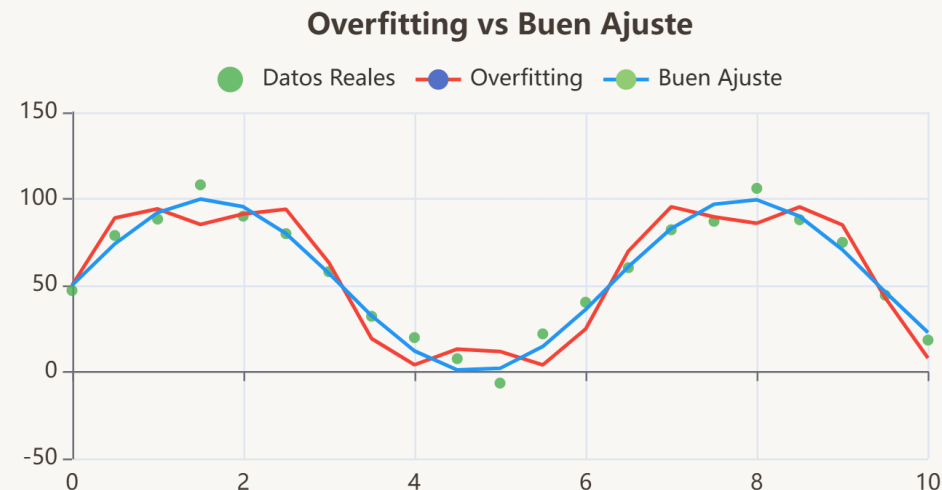
random_state

Semilla para reproducibilidad. Con un valor fijo (ej. 42), siempre se obtiene la misma división.

shuffle

Mezcla los datos antes de dividir (por defecto True) para asegurar una muestra aleatoria.

Visualización: Overfitting vs Buen Ajuste



Overfitting

- Bajo error en training
- Alto error en test
- Memoriza, no aprende
- No generaliza bien

Buen Ajuste

- Error similar en train/test
- Aprende patrones generales
- Generaliza bien
- Buena predicción

💡 **Mejor práctica:** Siempre divide tus datos. Es la única forma de saber si tu modelo realmente aprendió o solo memorizó.

Pandas: Manipulación de Datos en Python

Pandas es la librería fundamental para ciencia de datos en Python. Proporciona estructuras de datos de alto rendimiento y fáciles de usar.

¿Qué es un DataFrame?

Es una estructura de datos 2D (tabla) con filas y columnas etiquetadas. Es como Excel pero con el poder de Python.

Filas

Observaciones (estudiantes)

Columnas

Variables (Hours, Scores)

Operaciones Básicas

```
# Crear DataFrame df = pd.DataFrame(data)
# Explorar print(df.head()) # Primeras filas
print(df.shape) # Dimensiones
print(df.describe()) # Estadísticas
print(df.info()) # Info de tipos
```

df['col']

Seleccionar columna

df.loc[]

Filtrar por índice

df.iloc[]

Filtrar por posición

¿Por Qué Pandas?

-  **Rendimiento:** Optimizado para grandes datasets
-  **Versátil:** Limpieza, transformación, análisis
-  **Integración:** Compatible con otras librerías
-  **IO:** Lee/escribe CSV, Excel, SQL, JSON...

Flujo de Trabajo Típico

- 1 Cargar datos
- 2 Explorar (head, info)
- 3 Limpiar (valores faltantes)
- 4 Transformar (nuevas columnas)
- 5 Analizar (modelar)

 **Consejo:** Dedica tiempo a aprender pandas bien. Es la herramienta más importante para ciencia de datos en Python.

Scikit-learn: Machine Learning en Python

Scikit-learn (sklearn) es la librería de Machine Learning más popular de Python, construida sobre NumPy, SciPy y Matplotlib.

Estructura Modular

 **model_selection**: División de datos (train_test_split)

 **linear_model**: Regresión lineal, Ridge, Lasso

 **tree**: Decision Trees, Random Forest

API Consistente

```
# Todos los modelos siguen la misma estructura model
= Algoritmo() # Instanciar model.fit(X_train, y_train) #
Entrenar predictions = model.predict(X_test) # Predecir
score = model.score(X_test, y_test) # Evaluar
```

Funciones Usadas en Esta Práctica

train_test_split

Divide datos en conjuntos de entrenamiento y prueba

LinearRegression

Implementa regresión lineal ordinaria (OLS)

model.fit()

Entrena el modelo con los datos

model.predict()

Genera predicciones con datos nuevos

model.score()

Devuelve el coeficiente R^2 de determinación

Otros Algoritmos Disponibles

Regresión:

Ridge, Lasso, ElasticNet

Clasificación:

LogisticRegression, SVM

Árboles:

DecisionTree, RandomForest

Clustering:

KMeans, DBSCAN

Matplotlib: Visualización de Datos

Matplotlib es la librería de visualización más popular de Python. Es la base de otras librerías como Seaborn.

Elementos de un Gráfico

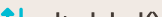
```
import matplotlib.pyplot as plt # Crear figura plt.figure(figsize=(10, 6)) # Datos plt.scatter(X, y, color='green', label='Datos') plt.plot(X, y_pred, color='orange', label='Predicción') # Elementos plt.title('Título del Gráfico') plt.xlabel('Eje X') plt.ylabel('Eje Y') plt.legend() plt.grid(True) # Mostrar plt.show()
```



plt.scatter()
Gráfico de puntos (dispersión)



plt.title()
Título del gráfico



plt.plot()



plt.plot()
Gráfico de líneas



plt.xlabel()
Etiqueta del eje X



plt.ylabel()

¿Por Qué Visualizar?

- Entender:** Ver patrones que no son obvios en números
- Explorar:** Identificar outliers y relaciones
- Comunicar:** Mostrar resultados de forma clara
- Validar:** Verificar que el modelo se ajusta bien

Otros Gráficos Útiles

plt.hist()

Histograma de distribución

plt.boxplot()

Diagrama de caja (outliers)

plt.bar()

Gráfico de barras



Tip: Combina matplotlib con seaborn para crear visualizaciones más atractivas con menos código.

Overfitting vs Underfitting: El Balance Correcto

⚠ Overfitting (Sobreajuste)

El modelo es **demasiado complejo** y memoriza los datos de entrenamiento, incluyendo el ruido.

Síntomas:

- Bajo error en training
- Alto error en test/validación
- Curva muy complicada

Causas:

Modelo muy complejo, pocos datos, entrenar por mucho tiempo.

Soluciones:

Más datos, simplificar modelo, regularización.

📈 Underfitting (Subajuste)

El modelo es **demasiado simple** y no captura los patrones importantes.

Síntomas:

- Alto error en training
- Alto error en test
- Curva muy simple

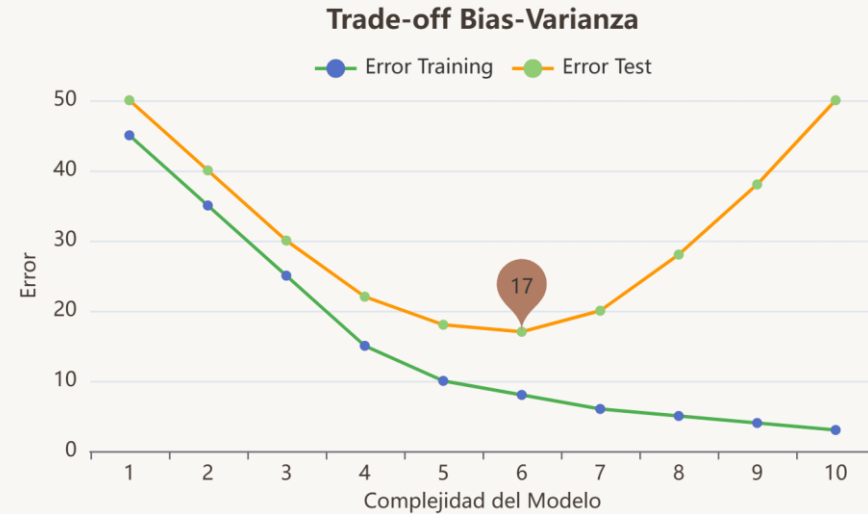
Causas:

Modelo muy simple, poco entrenamiento, mala selección de features.

Soluciones:

Modelo más complejo, más features, más entrenamiento.

Visualización del Balance



🎯 Balance Óptimo

El objetivo es encontrar el **punto medio** donde el modelo aprenda patrones sin memorizar ruido.

Características del balance óptimo:

- Error bajo en training y test
- Buena capacidad de generalización
- R^2 alto y consistente

💡 **Trade-off:** Reducir el bias aumenta la varianza y viceversa. El arte del ML está en encontrar el equilibrio.

Mejores Prácticas en Machine Learning

1 Divide Siempre los Datos

Usa train/test split o validación cruzada para evaluación honesta. Nunca uses datos de test para entrenar.

2 Explora los Datos Primero

Antes de modelar, haz EDA: visualiza distribuciones, busca outliers y entiende las relaciones entre variables.

3 Estandariza cuando Sea Necesario

Algunos modelos requieren features con escala similar. Usa StandardScaler (Estandarización) o MinMaxScaler (Normalización) cuando sea apropiado.

4 No Ajustes con Datos de Test

El test set es solo para evaluación final. Si ajustas el modelo viendo resultados de test, estás haciendo fuga de datos.

5 Documenta y Versiona

Documenta tu código, datos y experimentos. Usa control de versiones (Git). Reproducibilidad es clave en ciencia.

6 Usa random_state (Estado de aleatoriedad) para Reproducibilidad

Fija semillas aleatorias (ej. random_state=42) en train_test_split y modelos para obtener resultados consistentes.

7 Evalúa Múltiples Métricas

No te fíes de una sola métrica. Para regresión, mira R^2 (Coeficiente de Determinación), MAE (Error Absoluto Medio), RMSE (Raíz del Error Cuadrático Medio) y visualiza residuales.

8 Comienza Simple, Luego Compleja

Empieza con modelos simples (regresión lineal) como línea de base. Solo usa modelos complejos si justifican el costo.